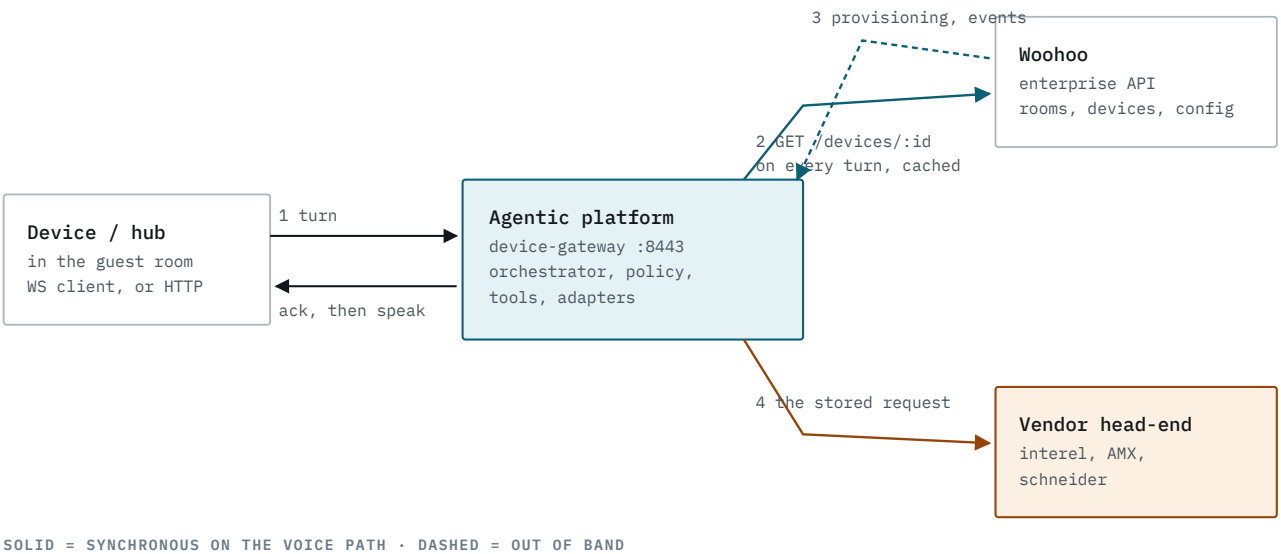
Woohoo V2 API Specification

How the in-room device, the Woohoo enterprise API and the agentic platform talk to each other. Every endpoint below is implemented and running; the calling direction of each one is stated explicitly, because that is where the wiring is usually misread.

3 integration directions **6** platform endpoints **7** catalog endpoints you implement
6 WebSocket frames **11** events

01 Who calls whom

There are three integration directions, and they are not symmetric. The most common misunderstanding is assuming the enterprise API pushes into the agent — it does not. The agent *pulls* room and device records from it, on demand, as the source of truth.



Direction 2 is the one people invert. The platform holds no permanent room database of its own when an enterprise API is configured — it asks yours, every turn, and caches the answer.

#	CALLER	CALLEE	TRANSPORT	WHEN
1	Device / hub	Platform	WebSocket or HTTP	Every guest utterance
2	Platform	Woohoo enterprise API	HTTP JSON	Device resolution, cached 60 s
3	Woohoo enterprise API	Platform	HTTP + SSE	Provisioning a room; consuming events
4	Platform	Vendor head-end	HTTP	When a turn actuates something

02 Device to platform

Two ways in, same orchestrator behind both. WebSocket is the device path: one connection per hub, held open, so the platform can send a filled pause while the turn runs. HTTP is the one-shot path for curl, tests and non-voice callers.

WS `ws://host:8443/` — the device path

Six frame types. `hello` must be first; anything else before it is rejected with `NOT_AUTHENTICATED`.

CLIENT → SERVER

```
{ "type": "hello",  "deviceId": "d_8bf3edc84290edc3",
                    "firmware": "1.4.2", "certificateSerial": "..." }

{ "type": "turn",   "transcript": "turn off the bedroom light",
                    "asrConfidence": 0.95 }
```

SERVER → CLIENT

```
{ "type": "ready",  "property": "WooHoo Hotel", "room": "549",
                    "assistantName": "Woohoo", "phrasePackVersion": "v3" }

{ "type": "ack",    "filledPause": "one moment" }

{ "type": "speak",  "text": "I've asked for that light change.",
                    "turnId": "turn_mtps9d0l0" }

{ "type": "error",  "code": "BAD_FRAME", "detail": "Frame was not valid JSON." }
```

Order of events: `hello` → `ready` → `turn` → `ack` (immediately) → `speak` (when the turn completes).

POST `/v1/turn` — one-shot equivalent

The field is `transcript`, not `text` — a common 400. Body is capped at 16 KB.

REQUEST

```
curl -X POST http://localhost:8443/v1/turn \
  -H 'Content-Type: application/json' \
  -d '{"deviceId":"d_8bf3edc84290edc3",
       "transcript":"turn off the bedroom light",
       "asrConfidence":0.95}'
```

RESPONSE, TRIMMED

```
{
  "turnId": "turn_mtpt8ndi1",
  "speech": "I've asked for the bedroom light to be turned off.",
  "lane": "agentic",
  "escalated": false,
  "awaitingConfirmation": false,
  "totalMs": 2196,
  "costUsd": 0.000691,

  "calls": [
    { "tool": "lighting.set", "tier": "T1",
      "verdict": "ALLOW", "ruleId": "allow", "ok": true }
  ],

  "vendorCalls": [
    { "vendor": "masterdb-catalogue/schneider",
      "method": "PUT",
      "url": ".../objects/79742578/properties/85",
      "objectId": "79742578",
      "capability": "lighting.on_off", "zone": "bedroom",
      "requested": { "anyOfValue": [ { "type": "unsigned_integer", "value": 5 } ] },
      "status": 200, "ok": true, "ms": 58,
      "note": "row 35 - Bedroom Off" }
  ],

  "guardrails": [ /* 9 controls, what each did this turn */ ],
  "pipeline":   [ /* per-stage timings */ ]
}
```

`vendorCalls` is the audit answer to "what did it actually send?" — one entry per request that reached a head-end, with the exact body.

There is no authentication on this endpoint today. `deviceId` is a field in the body; `certificateSerial` exists on the device record and nothing verifies it. Anyone who can reach the port can claim any room. Fine for a LAN test rig, not for a property.

03 Platform to Woohoo enterprise API

This is the contract **you implement**. Set `WOOHOO_CATALOG_URL` and the platform stops using its own room store entirely: every device resolution becomes an HTTP call to your service. Seven endpoints, all JSON.

Reads are cached for 60 seconds because device resolution happens on every single turn and a room record changes roughly never. Writes invalidate the cache immediately.

METHOD	PATH	RETURNS	CALLED
GET	{base}/devices/:deviceId	DeviceRecord	Every turn
GET	{base}/rooms/:roomId	RoomRecord	Every turn
GET	{base}/properties/:propertyId	PropertyConfig	Every turn
GET	{base}/properties/:id/rooms	RoomRecord[]	Admin listing
PUT	{base}/devices/:deviceId	2xx	On provisioning
PUT	{base}/rooms/:roomId	2xx	On provisioning
PUT	{base}/properties/:propertyId	2xx	On config change

GET {base}/devices/:deviceId

the hot path

YOU RETURN

```
{
  "deviceId":           "d_8bf3edc84290edc3",
  "tenantId":           "t_park_royal",
  "propertyId":         "p_woohoo_hotel_167",
  "roomId":             "r_167_549",
  "certificateSerial":  "imported:8bf3edc84290edc3",
  "boundAt":            1757000000000,
  "bindingVerifiedAt":  1757000000000,
  "revoked":            false
}
```

GET {base}/rooms/:roomId

YOU RETURN

```
{
  "roomId":       "r_167_549",
  "propertyId":   "p_woohoo_hotel_167",
  "roomNumber":   "549",
  "roomTypeId":   "rt_smart_room_1",
  "vendorRoomId": "2389"
}
```

`vendorRoomId` is not the room number. It is the identifier the automation vendor knows — in the V1 database that is `rooms.id`. Assuming the two match is a classic onboarding bug whose symptom is actions landing in the wrong room.

> **404 and failure mean different things.** A 404 is read as “not configured” and returns `undefined`. Any other failure throws `CatalogUnavailable` — because silently treating an outage as “no such room” would revoke a working device mid-stay.

Configuration:

```
WOOHOO_CATALOG_URL=https://enterprise.heywoohoo.com/v1/catalog
WOOHOO_CATALOG_TOKEN=<sent as Authorization>
```

04 Woohoo enterprise API to platform

Two ways in: push a room into the catalog at runtime, or subscribe to the event feed and react to what the agent did.

POST `/v1/admin/rooms` — needs x-admin-token

Creates a room and optionally binds a device. Supply either a `roomTypeId` or an explicit capability list — not neither.

REQUEST

```
curl -X POST http://localhost:8443/v1/admin/rooms \
  -H 'Content-Type: application/json' \
  -H 'x-admin-token: gen_55e1f36fe17d9b4c55a23b7a' \
  -d '{
    "roomNumber":   "612",
    "vendorRoomId": "3410",
    "roomTypeId":   "rt_smart_room_1",
    "deviceId":     "d_9ac41f22b7e05d81",
    "propertyId":   "p_woohoo_hotel_167"
  }'
```

RESPONSE

```
{ "roomId": "r_612", "roomNumber": "612", "vendorRoomId": "3410",
  "roomTypeId": "rt_smart_room_1", "deviceId": "d_9ac41f22b7e05d81",
  "propertyId": "p_woohoo_hotel_167" }
```

The token is printed at startup. Set `WOOHOO_ADMIN_TOKEN` to pin it, otherwise it is regenerated on every restart.

SSE `/v1/events` — every turn, from any source

SUBSCRIBE

```
curl -N http://localhost:8443/v1/events

data: {"kind":"turn.received","turnId":"turn_abc","transcript":"..."}

data: {"kind":"turn.completed","turnId":"turn_abc","speech":"...",
       "calls":[...],"vendorCalls":[...]}
```

Turns submitted over HTTP, WebSocket or from another service all appear here, so a staff dashboard sees everything without polling.

DOMAIN EVENT	RAISED WHEN
`woohoo.turn.completed`	Every turn, including refusals
`woohoo.policy.denied`	A proposed tool call was blocked
`woohoo.escalation.raised`	Needs a human, with an acknowledgement deadline
`woohoo.service_request.created`	Housekeeping or maintenance dispatched
`woohoo.service_request.updated`	Status moved
`woohoo.fnb_order.placed`	Order confirmed after the confirmation turn
`woohoo.device_state.changed`	Room state moved
`woohoo.device.online / .offline`	Hub connectivity
`woohoo.stay.checked_out`	Triggers guest-data erasure
`woohoo.capability_manifest.updated`	A room type gained or lost a capability

05 Inspection endpoints

Read-only, no auth. Useful for health checks, dashboards and debugging a property.

GET /health

How you tell a mock deployment from a real one at a glance.


```
{ "status":"ok", "connections":0, "uptimeS":13306,
  "provider":"gemini", "model":"gemini-2.5-flash",
  "adapter":"masterdb-catalogue" }
```


GET /v1/devices

Every device currently loaded, with the room it resolves to.


```
{ "devices": [
  { "deviceId":"d_8bf3edc84290edc3", "roomId":"r_167_549",
    "roomNumber":"549", "roomTypeId":"rt_smart_room_1",
    "vendorRoomId":"2389", "propertyName":"WooHoo Hotel",
    "revoked":false }
] }
```


GET /v1/rooms/:deviceId/devices

What is physically in that device's room, and the endpoint behind every state.


```
{ "room": { "roomNumber":"549", "vendorRoomId":"2389" },
  "devices": [
    { "item":"Bedroom", "zone":"bedroom", "kind":"toggle",
      "capability":"lighting.on_off", "vendors":["schneider"],
      "inert":0,
      "endpoints":[
        { "id":34, "state":"Bedroom On",  "value":true,
          "method":"PUT", "url":".../objects/79742578/properties/85",
          "body":{ "anyOfValue":[{"type":"unsigned_integer","value":6}] },
          "inert":false,
          "origin":{ "apiUrl":"https://<vendor host>/...",
                     "headerNames":["Authorization"] } }
      ] }
  ],
  "unmapped": [ { "item":"MuR", "reason":"housekeeping request, not a device" } ] }
```


`origin` shows where the command points in production; header *names* only, never values.

06 Databases

Two schemas, different jobs. The V1 `masterdb` is read as a command catalogue — it already holds a finished HTTP request for every state every device can be put into. The V2 schema holds the platform's own records.

V1 masterdb — the command catalogue

TABLE	ROWS	USED FOR
roomAutomationServices	14,312	One stored HTTP request per device state
rooms	2,610	Room number, hub binding, property
hubs	1,717	The in-room voice unit, becomes a device record
propertyDetails	74	Hotel, timezone, domain
domains	55	Brand, becomes the tenant
interels	93	Per-hub vendor project binding
roomAutomationServiceOrigins	14,192	Added: production URLs, header names only

The row the adapter sends, unchanged:

```
SELECT "apiMethod", "apiUrl", "apiHeaders", "apiBody"
  FROM "roomAutomationServices"
 WHERE "roomId" = 2389          -- the vendorRoomId
   AND "itemName"  = 'Bedroom'
   AND "stateValue" = 'false'
   AND "deletedAt" IS NULL;

-- PUT  .../v1/protocols/bacnet/local/objects/79742578/properties/85
-- {"anyOfValue":[{"isArray":false,"type":"unsigned_integer","value":5}]}
```

V2 schema — platform records

TABLE	HOLDS	NOTES
tenant / property	Brand and hotel, with the whole PropertyConfig as JSONB	Row-level security by tenant
room_type	Capability manifest per type	Manifest attaches to type, not room
room	Room number, type, `vendor_room_id`	Unique per property + number
device	Certificate serial, room binding, revoked flag	Separate entity from the room, bound revocably
service_request / fnb_order	Operations raised by a turn	
guest_data.*	Stay, consent, preferences	Own schema, per-tenant key, surgical erasure
turn_record	Append-only audit of every turn	`REVOKE UPDATE, DELETE`
escalation	Raised, acknowledged	Alarm on unacknowledged

> **The V2 schema is defined but not wired up.** `db/migrations/001_init.sql` exists; the running platform keeps catalog, sessions and audit in memory, so a restart loses them. Point the catalog at your enterprise API and that problem moves to your database, which is the intended answer.

07 Errors

Two vocabularies. HTTP codes are transport-level; tool error codes are what the agent is told, and are deliberately a small closed set so the model cannot be confused by vendor strings.

HTTP ERROR	MEANING
BAD_REQUEST	Missing `deviceId` or `transcript`, or malformed JSON
PAYLOAD_TOO_LARGE	Body over 16 KB
UNAUTHORISED	Missing or wrong `x-admin-token`
NOT_FOUND	No such route, device or room
METHOD_NOT_ALLOWED	Wrong verb for the path
INVALID_ROOM	Provisioning payload failed validation
PROVISION_FAILED	Room could not be created

TOOL ERROR	MEANING
NOT_AVAILABLE	This room cannot do that — no such capability or device
NOT_PERMITTED	Policy refused it
NEEDS_CONFIRMATION	Tier 3 — requires a separate spoken confirmation turn
INVALID_ARGUMENT	Failed strict schema validation
DUPLICATE	Same request already in flight or just completed
RATE_LIMITED	Too many of these in the window
TEMPORARILY_UNREACHABLE	Head-end down or timed out — retryable

08 Connect in five minutes

The shortest path from nothing to a command reaching a head-end.

```
# 1 - a head-end to receive commands
npm run head-end                                  # :8092

# 2 - the platform, pointed at the V1 catalogue
GEMINI_API_KEY=...            \
WOOHOO_ADAPTER=masterdb       \
WOOHOO_SEED_PROPERTY=all      \
npm run gateway                                   # :8443

# 3 - confirm it is live and real
curl -s localhost:8443/health

# 4 - find a device to speak as
curl -s localhost:8443/v1/devices | jq '.devices[0]'

# 5 - send a turn
curl -s -X POST localhost:8443/v1/turn \
  -H 'Content-Type: application/json' \
  -d '{"deviceId":"d_8bf3edc84290edc3","transcript":"turn off the bedroom light"}' \
  | jq '{speech, vendorCalls}'

# 6 - see what the head-end received
curl -s localhost:8092/_captured | jq '.requests[0]'
```

> **To put your own service behind it**, implement the seven endpoints in section 03 and set `WOOHOO_CATALOG_URL`. Nothing above that boundary changes — the platform does not care where room and device records live.